Int. J. Human–Computer Studies 184 (2024) 103204



Contents lists available at ScienceDirect

International Journal of Human - Computer Studies

journal homepage: www.elsevier.com/locate/ijhcs





Leyenes: A gaze-based text entry method using linear smooth pursuit and target speed

Piercarlo Dondi, Samuel Sapuppo, Marco Porta *

University of Pavia, Department of Electrical, Computer and Biomedical Engineering, Via A. Ferrata 5, Pavia, 27100, Italy

ARTICLE INFO

Keywords:
Eye tracking
Gaze writing
Gaze interaction
Text entry
Smooth pursuit
Interface design
Accessibility
User study

ABSTRACT

Gaze-based writing is one of the most widespread eye tracking applications for human–computer interaction. While eye tracking communication has traditionally been employed as an assistive technology, declining prices of eye trackers now make it a feasible alternative to keyboards or touchscreens in many contexts (for example, the interaction with public info points). In this paper we propose Leyenes, a text entry method based on smooth pursuit, a natural eye movement that occurs when the gaze follows a moving target. Our approach requires no explicit calibration by the user, allowing for more spontaneous interaction and enabling eye input even when calibration is difficult to achieve or maintain. To the best of our knowledge, Leyenes is the first text entry technique based on smooth pursuit that considers both (approximate) gaze speed and position and employs a linear interface instead of the more common circular layouts. The results of the user study we conducted show that the proposed solution is slow but robust, with a very low error rate, which makes it particularly suitable for extemporaneous writing of short text.

1. Introduction

Gaze-based input is a well-studied topic, with applications in numerous fields, such as web browsing (Kumar et al., 2017; Casarini et al., 2020), human–robot interaction (Shi et al., 2019; Di Maio et al., 2021), artwork analysis (Dondi et al., 2022), drawing (Huang et al., 2023), playing music (Davanzo et al., 2018; Valencia et al., 2019), controlling videogames (Istance et al., 2010), and writing (Porta, 2015; Cecotti, 2016; Rajanna et al., 2022), which is the focus of the present work.

This paper considers a specific category of gaze-based writing techniques, namely methods based on *smooth pursuit*, a natural eye movement that occurs when the gaze follows a moving target (Vidal et al., 2013). The proposed solution, called Leyenes, does not need any explicit calibration, allowing for a more spontaneous interaction in which the user can simply approach the eye tracking device and start the communication process.

Even if methods based on smooth pursuit are normally robust to user movements, the absence of calibration inevitably leads to a reduction in precision and speed. Calibration-free gaze-based writing systems are significantly slower than calibrated solutions, which are instead more suitable for prolonged use — the time needed for the calibration procedure becomes negligible for the user. Traditionally, eye tracking technology has been used in the assistive context for motor impaired people (Majaranta et al., 2011; Perfect et al., 2020), where a prolonged use is more common. However, researchers have also considered eye tracking as a way to interact with public displays (Khamis et al., 2015; Zeng et al., 2023), where an extemporaneous use for a short time is more probable (e.g., to specify the train destination in a ticket machine). In this scenario, an explicit calibration would be perceived as an unnecessary "nuisance" (and may be only partially useful, as the user may not stay perfectly still during the whole interaction process). Leyenes is mainly conceived as a writing technique for short text entry.

Leyenes employs a linear layout with horizontal and vertical lines and takes into account both gaze position and speed (approximate, due to the absence of calibration). Our hypothesis was that using both these components the writing performance can be improved, while maintaining a very low error rate. Robustness is a key factor for a gaze-based text entry method, as annoying typos (with the consequent need for frequent corrections) should be limited as much as possible, especially for applications designed for a short use.

To the best of our knowledge, Leyenes is the first text entry technique based on smooth pursuit that considers both approximate gaze speed and position and employs a linear interface instead of the more common circular layouts. The present research aims at investigating, from both a theoretical and practical point of view, the potential of a new, unexplored configuration. For this reason, we also present a study we performed to evaluate how (approximate) gaze position and speed contribute to system performance and how many elements are

* Corresponding author.
E-mail address: marco.porta@unipv.it (M. Porta).

https://doi.org/10.1016/j.ijhcs.2023.103204
Received 7 April 2023; Received in revised form 31 October 2023; Accepted 12 December 2023
Available online 14 December 2023
1071-5819/© 2023 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

selectable using a linear design. We believe it is important to test the validity of alternative designs and input modalities in a still relatively little-studied field such as eye writing based on smooth pursuit. Since our reference application context is the extemporaneous input of short texts in public places, this work considers error robustness more relevant than writing speed.

2. Related work

The most common way of implementing gaze-based writing is through virtual on-screen keyboards, where keys are "pressed" when looked at for a specific *dwell time*. This approach is intuitive and fast – a theoretical speed of 22 words per minute has been demonstrated by Majaranta and Räihä (2007) – but has the disadvantage of requiring big graphical keys to prevent pointing precision issues. Several other solutions have therefore been developed which exploit non-standard key arrangements. For example, 12 large onscreen buttons are used in *GazeTalk* (Johansen et al., 2003), and the keyboard created by Cecotti (2016) is based on a tree selection approach with eight commands. Also *SliceType* (Benligiray et al., 2019) uses a keyboard, in the form of a circle inside a square. Variable dwell times have been taken into account by Panwar et al. (2012), Pi and Shi (2017), and Mott et al. (2017), as well as "eye switches", such as blinks or pressure pads (Li et al., 2009; Rajanna, 2016). Also blinking has been investigated (MacKenzie and Ashtiani, 2011).

As alternatives to virtual keyboards, numerous other methods have been developed. For instance, alphabet letters in *Dasher* (Ward et al., 2002) increase into large areas when observed, indicating the most likely subsequent letters. In *Stargazer* (Hansen et al., 2008), characters are arranged in a circle and gradually enlarge and move to the center when looked at. Letters are displayed in a two-layer pie menu in *EYEwrite* (Huckauf and Urbina, 2007), where looking into the primary circle makes a larger pie, containing all letters, appear. In order for these methods to function properly, calibration is necessary.

Another group of gaze-based text entry solutions take advantage of *gaze gestures*, that are gaze paths used to input characters or start certain processes (Heikkilä and Räihä, 2009; Porta, 2015). These writing techniques, while making the most of the available screen area, typically demand a greater initial effort from the user to learn and remember the gestures corresponding to the different characters and actions. Examples worth mentioning include *EyeWrite* (Wobbrock et al., 2008), *Eye-S* (Porta and Turina, 2008), and *Quikwriting* (Bee and André, 2008).

Hybrid solutions have also been considered. For instance, *EyeSwype* (Kurauchi et al., 2016) selects the initial and last letters of a word using "reverse crossing", while characters in the middle are selected by moving the gaze over the appropriate keys. Bafna (2018) describes a similar strategy. With *GlanceWriter*, Cui et al. (2023) analyze the user's gaze path and location to probabilistically determine the letters to be typed.

All of the writing strategies presented above require calibration to properly work. Very few calibration-free writing systems exist, and they are commonly based on smooth pursuit. This is consistent with the results of Vidal et al. (2013), Khamis et al. (2015), and Drewes et al. (2022), who showed that smooth pursuit is suitable to interact with a display without the need for calibration. *SMOOVS* (Lutz et al., 2015) can be regarded as the first writing system based on smooth pursuit, even if it exploits an initial one-point explicit calibration. Basically, six rotating hexagonal tiles (clusters) are arranged in a circle and include groupings of letters or symbols. The user's gaze path must match that of moving clusters and characters tiles. This method has influenced other approaches: Zeng and Roetting (2018) alter the original SMOOVS design by including a CNN word prediction function; Abdrabou et al. (2019) employ a circular cluster motion rather than a linear one; and Zeng et al. (2022) adopt a hexagonal interface with smaller clusters and introduce attention areas. *EyeTell* (Bafna et al., 2021), designed for the iPad's built-in camera, has an interface composed of two concentric rotating circles (one clockwise and the other counter-clockwise). *SPEye* (Porta et al., 2022) uses a sunburst layout in which each "beam" (radial straight line) contains a small circle moving from the center of the screen towards characters or character clusters. Due to the absence of calibration, the interfaces of all methods based on smooth pursuit need most of the screen area for them to work properly.

A study by Zeng et al. (2020), not directly related to writing, focuses on calibration-free interfaces based on smooth pursuit, with the main objective of determining the ideal number and speed of moving targets. Their experiments showed that, using a circular design and linear movements, 6–8 moving objects are better, even if the participants preferred a configuration with 12 elements. This analysis was a useful starting point for the design of our application.

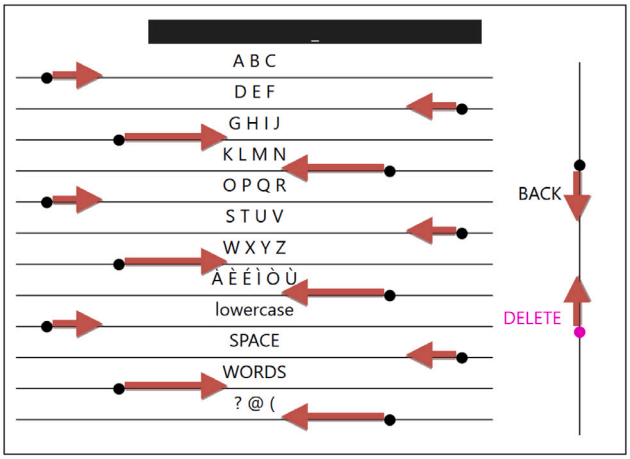


Fig. 1. Example of *Leyenes* screen (*Letters* screen) in light theme. The arrows indicate the moving direction of circles — short arrows denote a low speed (S_l), while long arrows indicate a high speed (S_h).

3. *Leyenes*: the proposed gaze-based text entry method

This section describes the main components and features of *Leyenes*. In particular:

- Section 3.1 presents the graphical user interface and the functionalities of the proposed system.
- Section 3.2 explains the selection principle of input elements and the algorithm behind it.
- Section 3.3 illustrates the hidden calibration method used to obtain a rough estimation of gaze position.

3.1. Interface and functions

3.1.1. Overview

The graphical user interface of *Leyenes*, available in the two themes "dark" (the default) and "light", is composed of a screen containing from one to twelve horizontal input elements and, when necessary, one vertical input element. A field containing the typed text is located at the top (Fig. 1).

An *input element* is a horizontal straight line containing a small moving circle or a vertical straight line containing two moving circles. Each input element has also an associated *label* indicating the corresponding action, character group, or single character.

To select the desired input element, the user must follow with their gaze the moving circle on the corresponding line. Circles on odd lines initially move from left to right, while circles on even lines initially move from right to left. When a circle reaches the end of a line, it reverses its direction. In screens with at least four horizontal lines, the

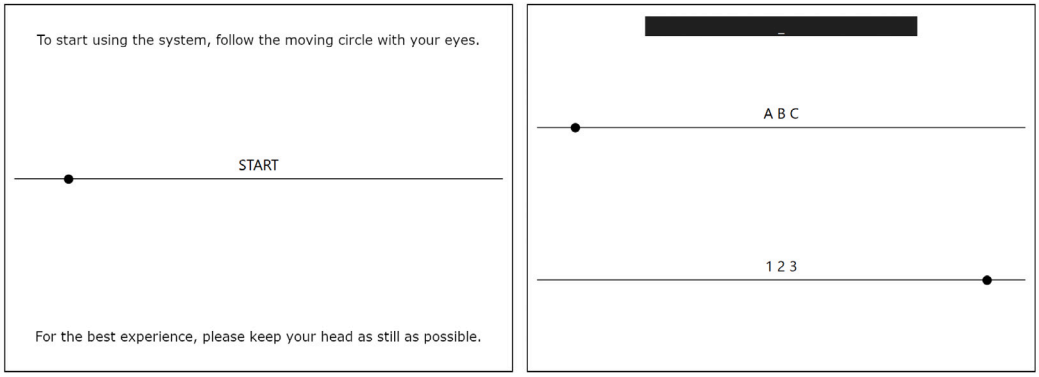


Fig. 2. *Initial* (left) and *Home* (right) screen.

circles of the first two lines move at speed S_l, while those on the next two lines move at speed S_h. The next possible lines follow the same pattern. In our experiments, S_l and S_h were, respectively, 112.5 px/s and 375 px/s, corresponding to about 3°/s and 9.9°/s at a viewing distance of 60 cm (the reference distance that we will use throughout the paper to report values in degrees).

The two circles on the vertical line, associated with the *Back* (to return to the previous screen) and *Delete* (to delete the last character entered) actions, move in opposite directions and are colored differently, to be distinguishable — they too reverse their direction once the end of the line is reached.

The size and color of the graphic elements, the number of lines to be displayed, and the speeds of circles can be set in a configuration file.

The smoothness of the movement of circles depends on the *motion step* frequency, that is the number of times the position of a circle is updated in one second. A good value for this parameter is half the sampling frequency of the employed eye tracker (since we used a 150 Hz device, in our experiments the motion step frequency was set to 75 Hz).

3.1.2. Screens and sub-screens

There are five types of screens: *Initial*, *Home*, *Letters*, *Numbers*, and *Symbols*. The *Initial* screen (Fig. 2, left) contains a single horizontal input element (*Start*) and shows short instructions on how to use the system. The selection of the *Start* element triggers the *Home* screen (Fig. 2, right). The *Initial* screen is automatically displayed after an idle time (during which no gaze is detected) of one minute. In this initial screen, we provide the suggestion to "keep the head as still as possible", for the best experience. While this is not essential for Leyenes to work properly, this is a good suggestion for any eye tracking application (people who have never used an eye tracking system may for example move a lot and get out of the field of view of the device).

From the *Home* screen, the *Letters* and *Numbers* screens can be reached. The circles on the two lines (*ABC* and *123*) on the *Home* screen move at the same speed (S_l) but in opposite directions.

The *Letters* screen (Fig. 1) consists of 12 horizontal lines: eight for groups of letters of the English alphabet and for accented letters, two for *Upper/Lowercase* and *Space*, one for symbols/punctuation marks, and one for word prediction (*WORDS*). Selecting a group of letters opens a sub-screen with group-specific characters (Fig. 3). Sub-screens have a vertical input element to return to the previous screen.

If the text box is empty, the label of the prediction line is *WORDS*, otherwise it is the list of the four most probable words (based on the letters entered so far, see Fig. 4). If the desired word is among the four shown, the corresponding sub-screen can be opened to choose the right one. Once selected, the word is added in the text box and the *Letters* screen is displayed again. If no matches with the words in the dictionary are found, the phrase "No words found" is shown as a label. Predictions can be adapted based on the application scenario — for example, in a shopping center the dictionary may simply contain the list of shops.

The *Numbers* screen (Fig. 5, left) contains eleven horizontal lines, one for each digit and one to access the *Symbols* screen.

The *Symbols* screen (Fig. 5, right) contains thirty-two of the most used symbols/punctuation marks, divided into eight groups of four characters each.

3.2. Element selection process

This section illustrates how the selection process works, from both the user's point of view (what they have to do and the visual feedback they receive, Section 3.2.1) and the system's perspective (how selections are detected, Section 3.2.2)

3.2.1. Selection principle

To select an input element, the user must follow with their gaze the moving circle on the corresponding line. When gaze direction and speed match a circle, its color changes to red (Fig. 6a). If the user's gaze is detected on the circle for a predefined time, then its color changes to green and its associated action is performed (Fig. 6b). When the circle reaches the end of its line, it reverses its direction and turns black again. If the user fails to select the chosen item before its turning point, they will have to start following the circle again.

After a selection, the system returns to the previous screen. If the user realizes that the currently red circle is not the correct one, they can look elsewhere: the color of the circle will become black again and no action will be performed. The whole selection process typically takes less than two seconds.

3.2.2. Selection algorithm

The selection algorithm consists of two phases: first, it checks if the user is performing a smooth pursuit; then, it verifies if there is a match between the user's gaze and the movement of one of the circles.

The selection of input elements should be robust and fast. Even a calibrated system is generally accurate but not very precise: by fixating a point on the screen, gaze samples are distributed close to the point, but not necessarily very close to each other. We therefore defined as *Point of Gaze* (PoG) the average x and y gaze coordinates computed over a certain number of consecutive samples. This value depends on the sampling rate of the eye tracker.

To determine if the user is trying to follow a moving circle, we firstly consider the gaze speed: if it is greater than zero and lower than the maximum circle speed (*max_circle_speed*), we assume that a potential smooth pursuit is occurring.

The (approximate) gaze speed is computed as follows. Given a group of collected PoGs, let *distance* be the horizontal distance between the first and last PoG, *sampling_frequency* the sampling frequency of the eye tracker (150 Hz in our case), *gaze_samples* the number of gaze samples

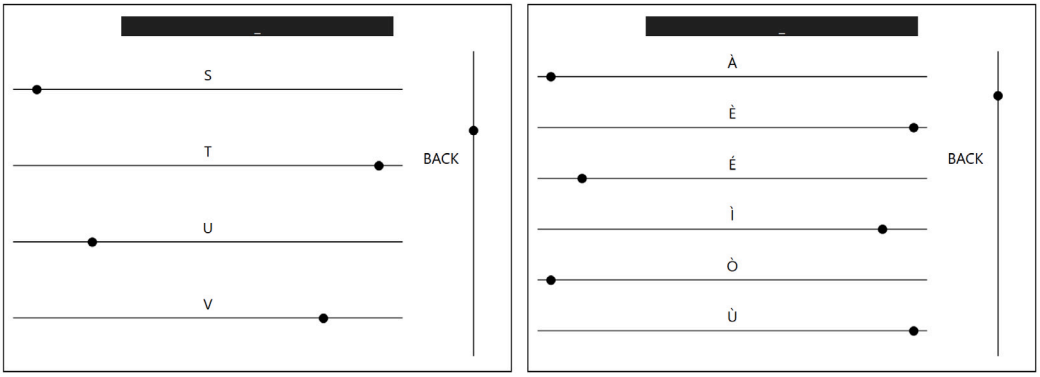


Fig. 3. Sub-screens opened by selecting the *STUV* group (left) and the accented letters group (right).

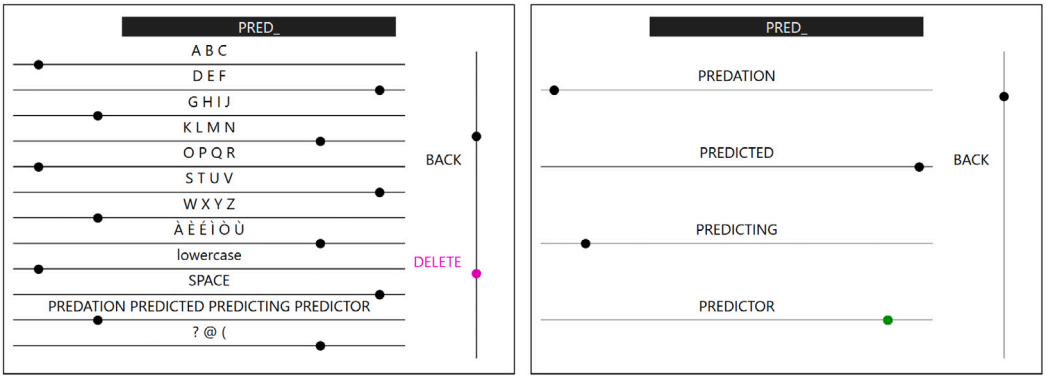


Fig. 4. Word prediction function: instead of the *WORDS* label, four words are displayed, based on the letters already entered (left); the desired word can then be selected in the corresponding sub-screen (right).

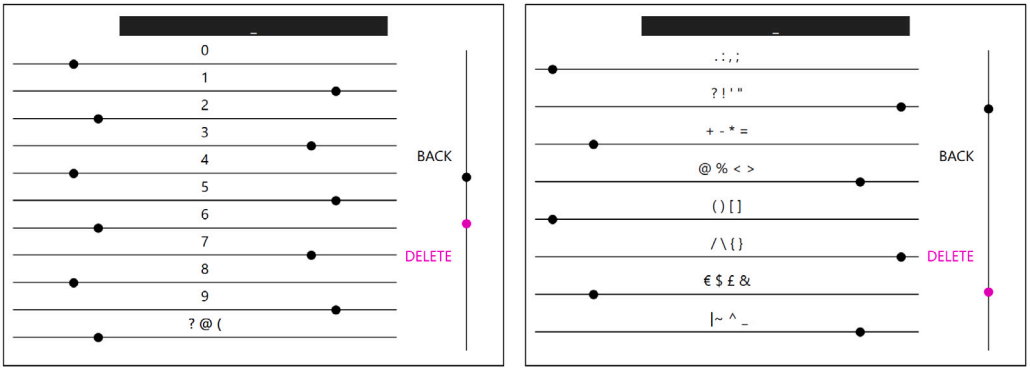


Fig. 5. *Numbers* (left) and *Symbols* (right) screens.

composing a PoG (20 in our experiments), and *n_PoGs* the number of collected PoGs; then, *gaze_speed* is given by Eq. (1).

$$gaze_speed = \frac{sampling_frequency}{gaze_samples} \cdot \frac{distance}{n_PoGs} \tag{1}$$

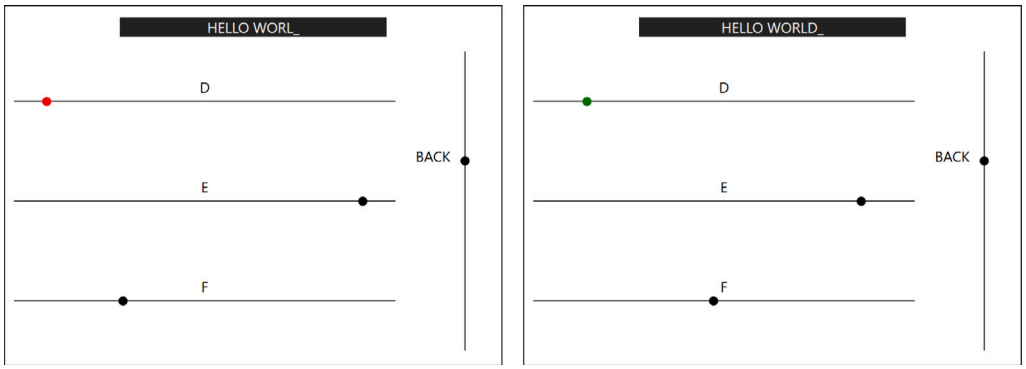
The algorithm starts by checking the speed condition ($0 < gaze_speed < max_circle_speed$) on the first three PoGs it receives. If the condition is not satisfied, the first two PoGs are discarded, the last one is kept, and the check is performed again once the next two PoGs are acquired. Otherwise, as a final confirmation of the smooth pursuit activity, a fourth PoG is checked.

Given a smooth pursuit, the identification of a match between the user's gaze and one of the circles depends on both the direction and speed of the gaze. Firstly, inspired by Drewes et al. (2019), the algorithm computes the regression line, which is defined by the considered PoGs using the linear least squares method; then, the type of gaze movement (*movement_type*, either horizontal or vertical) is discriminated as shown in Eq. (2), using the slope S of the line.

$$movement_type = \begin{cases} horizontal & \text{if } |S| \leq 0.8 \\ uncertain & \text{if } 0.8 < |S| < 1.2 \\ vertical & \text{if } |S| \geq 1.2 \end{cases} \tag{2}$$

If *movement_type* is *uncertain*, new PoGs are considered until a vertical or horizontal gaze movement is detected. Then, the system uses the collected PoGs to compute the *gaze direction*: from left to right (or vice versa) for a horizontal movement; from top to bottom (or vice versa) for a vertical movement.

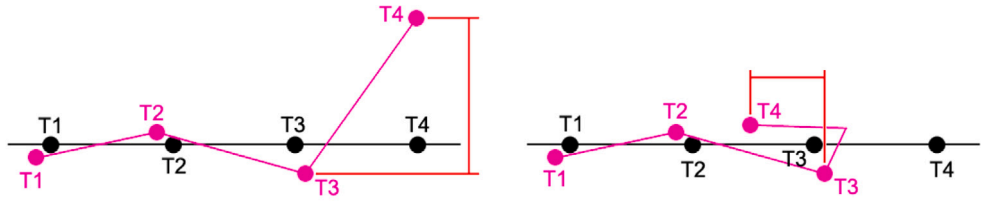
Each new PoG is checked against the current direction to verify whether it is coherent with it. If an abrupt change in direction is



(a) The system detects that the user has started following the circle associated with the letter 'D' and its color changes to red.

(b) Since the user keeps following the circle on the 'D' line, its color changes to green and the 'D' character is added to the text box.

Fig. 6. Selection of an input element.



(a) The gaze suddenly moves upwards.

(b) The gaze inverts direction.

Fig. 7. Possible changes of direction in the user's gaze. In black, the circle at four different times; in purple, the PoGs calculated at the same times.

detected (Fig. 7), meaning that the user has changed their object of interest, the collected PoGs are discarded and the algorithm is restarted.

A direction change is valid only if at least two consecutive PoGs in the opposite direction with respect to the direction of the previous PoGs are detected. If the direction is instead confirmed, the system proceeds to find a match with one of the input elements.

If the user's gaze is moving vertically, the algorithm discriminates the *Back* and *Delete* commands based on the match between gaze and circle direction. No speed check is necessary in this case since there is only one circle speed.

If the user's gaze is moving horizontally, the algorithm searches for the four closest input lines based on the y position (i.e., the average y coordinate of all the PoGs collected to confirm the direction). Then, it checks the speeds of the circles moving in the same direction.

When a circle matches with the user's gaze, an associated counter (initially set to zero) is increased by one. For counters greater than zero, the circle with the highest counter value is displayed in red (in case of equality, the firstly matched circle remains red). When a counter reaches a specific value (*num_matches*, three in our experiments), the corresponding circle turns green, and the selection of the associated input element occurs. If a circle reaches the end of its line before turning green, its counter resets, and it turns black again. Similarly, an abrupt change in the user's gaze direction resets all counters.

Overall, at least seven PoGs are required to perform a selection: four PoGs (*min_num_PoGs*) to verify whether the user is performing a smooth pursuit and to compute gaze direction and speed, another one to confirm the direction and get the first match, and other two to reach the number of matches (*num_matches*) needed to confirm a selection. Algorithm 1 summarizes the main steps of the selection procedure.

3.3. Hidden calibration

Without calibration, both the detected gaze speed and position may differ significantly from the actual speed and position of the target. Thus, while no explicit calibration is necessary for L*eye*nes, two types of (rough) "hidden" calibration are performed to enhance system performance. We consider these calibrations hidden because they occur while the user is interacting with the system, without being aware of them.

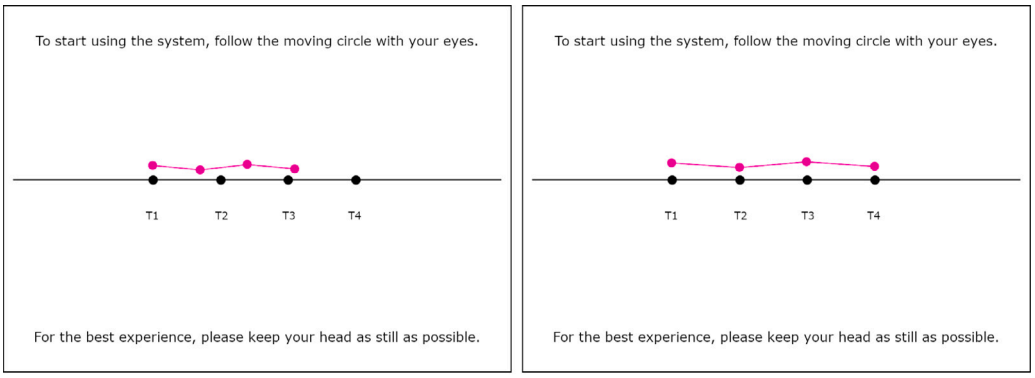
In the *Initial* screen (Fig. 2, left), the *Start* input element (the only one present) is vertically located in the middle of the screen. When hidden calibration is enabled, its selection requires the match counter to reach a value of five instead of three, so that more PoGs are collected. Once the *Start* element has been selected, the algorithm calculates the proportion between gaze and circle speeds. This value is used as a multiplying factor to adjust the gaze speed of the user (Fig. 8(a)). Similarly, the collected PoGs provide an estimate of the detected vertical position, allowing, if necessary, a vertical translation of the gaze (Fig. 8(b)).

Even if the detected gaze is in the right vertical position in the *Initial* screen (i.e., it is in the middle), this may not be the case for other vertical locations. For a better calibration, we therefore also use the *Home* screen (Fig. 2, right), which contains two lines: the selection of the *Letters* or *Numbers* elements allows to obtain an additional vertical reference.

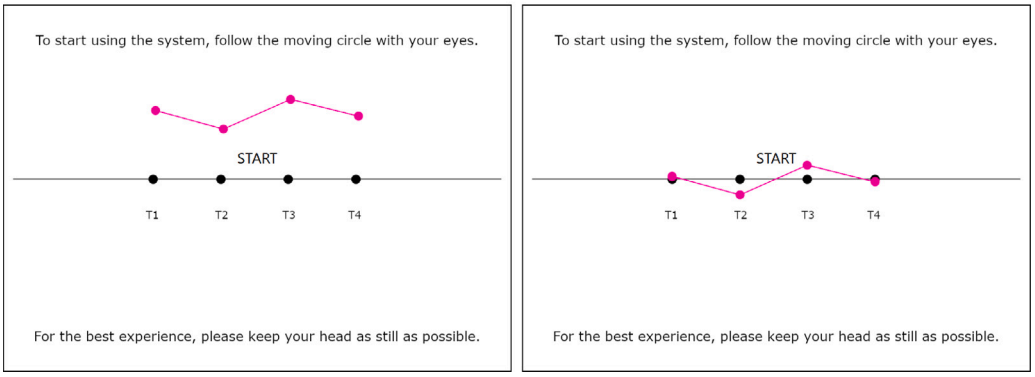
It is important to stress that this hidden calibration process can only provide a rough estimation of gaze position, useful for making some corrections but not sufficient to replace a complete and explicit calibration. Similar approaches are adopted by other writing methods based on smooth pursuit — e.g., the one-point calibration employed by SMOOVS (Lutz et al., 2015).

4. User study

Besides several informal evaluations, we conducted some formal tests to assess the performance of L*eye*nes. All the experiments were carried out in a quiet laboratory setting (Fig. 9), using a *Gazepoint GP3*



(a) Speed correction: without hidden calibration (left); with hidden calibration (right).



(b) Vertical position correction: without hidden calibration (left); with hidden calibration (right).

Fig. 8. Effects of hidden calibration on the detected gaze. The black circle is the target, while the purple circle indicates the estimated PoG.



Fig. 9. A participant in the user study.

HD eye tracker[1] (sampling rate 150 Hz) and a 24" monitor with full HD resolution (1920 × 1080 pixels). No explicit calibration was performed. The viewing distance was about 60 cm. Before each experiment, the participants were shortly explained how to use the writing system and could try it for three minutes. All participants signed an informed consent to the research.

[1] https://www.gazept.com/blog/introducing-the-gazepoint-gp3-hd/

4.1. Preliminary test

An initial test was carried out to determine the minimum circle speed under which it is not possible to distinguish a fixation from a smooth pursuit and the maximum speed that a circle can reach and still be reliably followed by the user's gaze. These goals are important both because the usability of the writing method depends on the efficient recognition of smooth pursuit and because we want the user experience to be as least "frustrating" as possible, thus keeping the number of errors very low.

We measured the circle speed in px/step, which is the number of pixels traveled by the circle in a single motion step (as explained in Section 3.1, the motion step frequency was set to 75 Hz).

The experiments were conducted with 12 participants aged between 20 and 61 (34 on average), 9 males and 3 females. Among them, 7 wore glasses, and one wore contact lenses. Five testers had previously tried eye tracking applications.

Each participant was instructed to follow a single circle moving from left to right along a horizontal line. Each time the circle reached the end of the line, it increased its speed by 0.5 px/step (equivalent to 1°/s) and restarted from the beginning. The process was repeated from an initial speed of 0.5 px/step to a final speed of 6.5 px/step (12.8°/s). We considered a circle "properly followed" if it was selected before the end of the line.

The results showed a minimum speed of 1.5 px/step (3°/s) for all participants. Regarding the maximum speed, most participants (8 out of 12) were able to correctly follow a circle moving at a speed up to 5.5 px/step (10.9°/s). Only one participant was able to follow the circle at 6.5 px/step (12.8°/s). Since the speed of 5 px/step (9.9°/s) was reached by 11 participants, we considered it as the "best" maximum speed, even

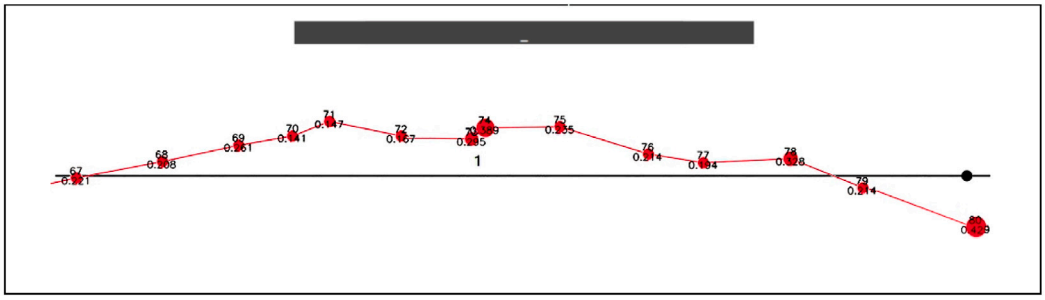


Fig. 10. Example of gaze movement along a horizontal line.

Algorithm 1: Element Selection Procedure.

```
Initialize PoG_List
while PoG detected do
    if size of PoG_List < min_num_PoGs then
        Add PoG to PoG_List
    else if getGazeSpeed(PoG_list) > max_circle_speed then #using Eq. (1)
        Empty PoG_list
    else
        slope ← getLineSlope(PoG_list)  #using linear least squares method
        movement_type ← getMovementType(slope)  #using Eq. (2)
        if movement_type is "uncertain" then
            Add PoG to PoG_List
        else
            direction ← getGazeDirection(PoG_list)
            isChanged ← checkDirectionChanges(direction, PoG)
            if isChanged then
                Empty PoG_list
            else
                Add PoG to PoG_List
                if movement_type is "vertical" then #gaze is moving vertically
                    input_element ← findMatch(direction)
                else  #gaze is moving horizontally
                    y_position ← getYGazePosition(PoG_list) #average of y coords
                    speed ← getGazeSpeed(PoG_list)  #using Eq. (1)
                    input_element ← findMatch(direction, y_position, speed)
                end if
                Increment input_element_counter
                if input_element_counter is num_matches then
                    return input_element  #input element found
                end if
            end if
        end if
    end if
end while
```

if 5.5 px/step can also be considered acceptable. A similar experiment for the vertical line, whose length is half the horizontal one, showed a maximum speed of 2.5 px/step (5°/s).

A problem emerged from the analysis of the collected gaze data. Due to the absence of calibration, smooth pursuits on a circle moving along a horizontal line were not linear but tended to move upwards when approaching the middle of the screen (Fig. 10). Such gaze shifts may be confused with changes in the element of interest. Considering all the tests and participants, the maximum vertical displacement measured with respect to a horizontal line was 226 px (about 6°); we therefore used this value as a threshold to verify whether a PoG is still in an acceptable position when performing a smooth pursuit on a horizontal line. As regards vertical lines, the problem was much less pronounced and did not require specific corrections.

4.2. Performance study

We designed some tests to identify the best number of horizontal input elements and circle speeds. Hidden calibration (Section 3.3) was always enabled. Given the peculiar characteristics of Leyenes, finding the right compromise between the number of input elements (the more they are, the faster writing becomes, because fewer intermediate screens are needed) and the reliability of the text entry process (lines too close to each other increase the number of errors) is essential to improve the user experience.

The experiments were conducted with 33 participants (including 11 participants from the preliminary test), aged between 16 and 73 (36 on average), 22 males and 11 females. Among them, 19 wore glasses and 4 wore contact lenses. Twelve testers had previous experience with eye tracking applications. In the end, the data of two participants were excluded from the analysis because of their insufficient quality — probably due to strong corrective glasses and/or bluish eyes, which are known issues of eye tracking technology (Nyström et al., 2013).

4.2.1. Number of evenly distributed lines

With this experiment we wanted to find the maximum number of horizontal lines that can be reliably used in a single screen. We started with four lines and the two speeds 112.5 px/s (3°/s) and 375 px/s (9.9°/s), where two circles with the same speed moved in opposite directions (Fig. 11, left). The participant was asked to select all the input elements. Once this operation was completed, four new lines were added, until 24 (Fig. 11, right) — beyond this value the elements would graphically overlap on the screen. The new lines always had the same two speeds.

Fig. 12 shows the number of participants who successfully completed the test with the different numbers of lines. All participants were able to perform selections with up to 12 lines, 94% of them (i.e., 29 out of 31) were able to use 16 lines, and 39% (i.e., 12 out of 31) reached 24 lines.

Since the accuracy of the Gazepoint GP3 HD eye tracker ranges from 0.5° to 1°, the worst case of 1° corresponds to about 38 pixels on our 24" full HD screen at a viewing distance of 60 cm. This means that, avoiding lines very close to the upper and lower edges, a calibrated system would roughly allow to reliably use up to 26 lines. Since we can have a maximum of 24 lines in our interface (due to the presence of other elements at the top of the screen), a calibrated system would allow all lines to be selected.

4.2.2. Number of evenly distributed speeds

This test was similar to the previous one (Section 4.2.1), but every time the number of lines was increased, also the number of speeds changed: we wanted to determine how many speeds can be employed at the same time.

We started with four horizontal lines and the two speeds 112.5 px/s (3°/s) and 412.5 px/s (10.9°/s), where two circles with the same speed moved in opposite directions. Once the participant correctly selected all the input elements, two new lines were added, with their circles moving

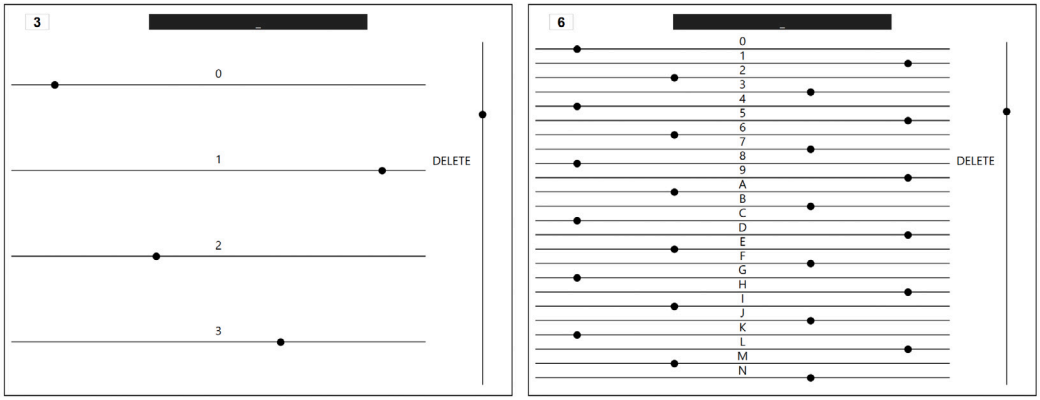


Fig. 11. Test on the number of evenly arranged lines — Initial (left) and final (right) experimental conditions.

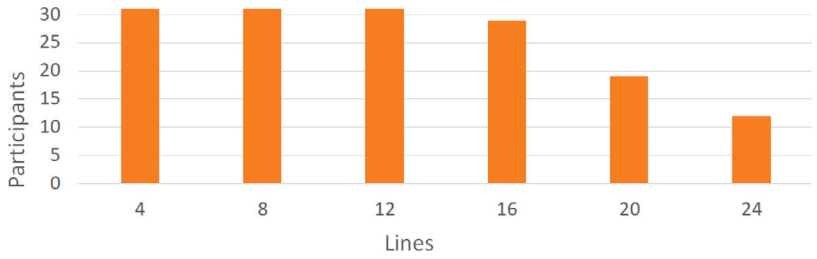


Fig. 12. Test on the number of evenly arranged lines (number of participants who completed the test with the different numbers of lines — two speeds only).

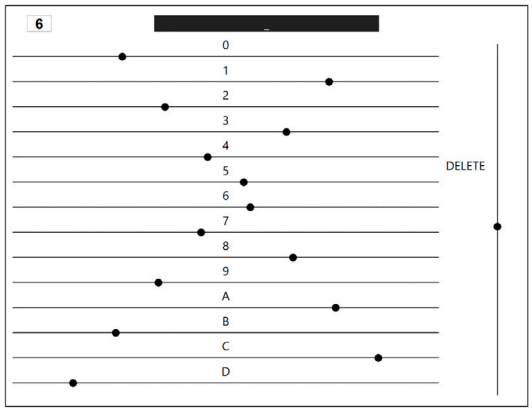


Fig. 13. Test on the number of evenly distributed speeds — Example with seven speeds, between 112.5 px/s and 412.5 px/s.

at the same speed but in opposite directions. The speeds in the whole screen were updated to be equally distributed between the two speeds 112.5 px/s and 412.5 px/s. This process continued until the participant made three consecutive selection errors. Fig. 13 shows an example with seven speeds (i.e., 14 lines).

Fig. 14 summarizes the results of the test: all participants were able to correctly perform selections with up to two speeds (4 lines) and the 83% (26 out of 31) with up to three speeds (6 lines); the performance started decreasing at four speeds (8 lines), and only one participant was able to reach eight speeds.

Overall, considering the results of this test and of that presented in Section 4.2.1, we decided to use twelve lines and two speeds (at most) as a standard configuration for Leyenes. This choice allows to have groups of four characters, with the only exception of accented letters (with five characters).

4.3. Speed and position contribution

With these tests we wanted to assess the degree to which speed and position contribute to system performance. In a writing system like Leyenes, in which both location and speed of the moving target (even if roughly evaluated through a hidden calibration) contribute to the identification of the specific input, we deemed it important to investigate the extent of the contribution of each component. We therefore repeated the previous experiments disabling either the check on speed (i.e., using only one speed) or the check on position. We considered both the case with hidden calibration and the case without.

The same participants of the performance study (Section 4.2) were involved in these tests.

4.3.1. Number of evenly distributed lines without speed reference

This test was aimed at evaluating the performance of Leyenes based exclusively on the detected gaze position. The direction and speed of circles was the same for all lines: from left to right at 187.5 px/s (5°/s), i.e., half the standard maximum speed. The experiment began with two horizontal lines (Fig. 15, left) and then, one at a time, new lines were progressively added, up to 18 (Fig. 15, right). The lines were evenly arranged in the available space.

Fig. 16 summarizes the outcome of the experiment. With hidden calibration, all participants were able to use up to five lines and 90% of them (28 out of 31) up to six. About half of the participants were able to use up to nine lines, but from ten lines the performance started to drop (only one participant succeeded with 15 lines). The results without calibration were significantly worse, with 90% of participants only managing to use up to three lines, and 60% four lines. Three participants were not even able to use two lines. Only one participant reached 18 lines.

Comparing these results with those in Fig. 12, where two speeds were used, we can notice a significant decrease in performance, even with hidden calibration (only 5 lines selected by all participants, instead of 12): this clearly means that speed plays a significant role in our selection algorithm.

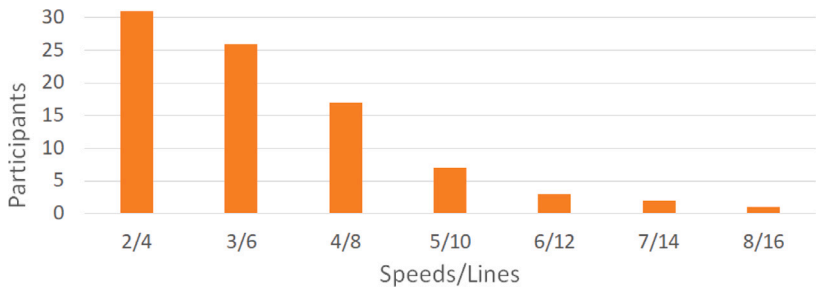


Fig. 14. Test on the number of evenly distributed speeds — Number of participants who completed the test for each number of speeds/lines.

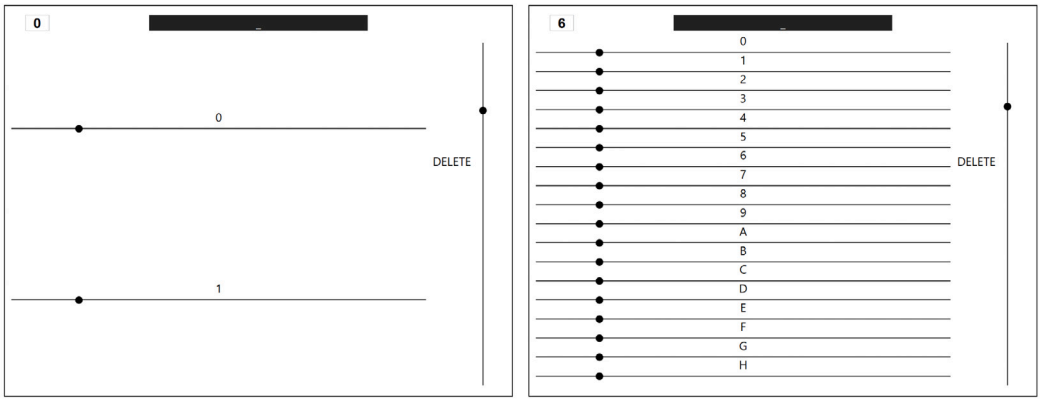


Fig. 15. Test on evenly distributed lines without speed reference — Initial (left) and final (right) experimental conditions.

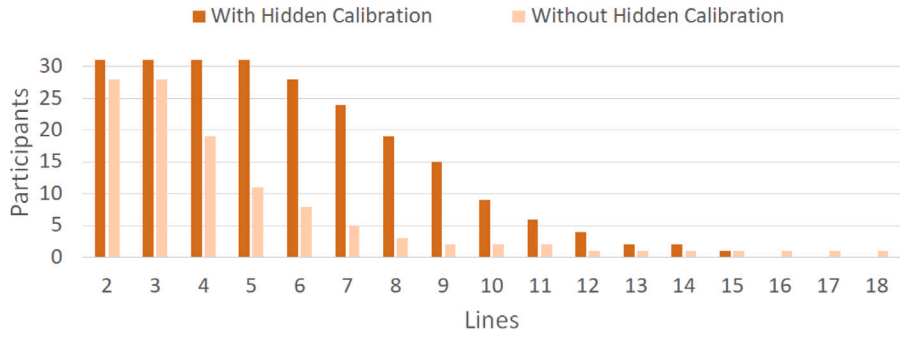


Fig. 16. Test on evenly distributed lines without speed reference — Number of participants who completed the test for each number of lines, with and without hidden calibration.

4.3.2. Number of evenly distributed speeds without position reference

With this experiment we wanted to evaluate the behavior of the system when the selection algorithm only exploits gaze speed, with no information about gaze position. The structure of the test was the same as that presented in Section 4.2.2.

Comparing the results in Fig. 17 with those in Fig. 14, we can see that the maximum number of speeds has now dropped from eight to six using hidden calibration, and to four without calibration. Using hidden calibration, the number of participants who could use two or three speeds is close to that of Fig. 14, while without hidden calibration the number of participants who could use three speeds drops from 24 to 6. An estimation of the vertical gaze position, even if implemented in a basic way with a hidden calibration, is therefore important for our input method.

4.4. Writing test

Of the 31 (valid) participants in the tests presented above, 24 (including 11 from the preliminary test) agreed to also participate in a "writing test". The participants' age was between 25 and 61 (36 on average), 15 males and 9 females. Among them, 14 wore glasses and three wore contact lenses. Seven participants had already used an eye tracker.

The purpose of this test was to evaluate the performance of Leyenes on two possible use cases: writing a sentence and entering a sequence of digits. These tasks reflect a double need. On the one hand, although Leyenes is not primarily designed to write long texts, we wanted to create an experimental structure comparable to that of other writing methods. On the other hand, with the mere writing of numbers we wanted to investigate the performance of Leyenes in a peculiar but not improbable use case.

We employed the default configuration, with 12 lines and two speeds. The word prediction option was disabled, to obtain objective performance results comparable to those of other gaze-based writing systems. Even if writing long texts is not the main goal of Leyenes, we considered a long sentence to obtain performance data comparable to other works.

The test was structured in two parts: firstly, the participant was asked to write a famous pangram (i.e., a sentence containing all the English alphabet letters), with an exclamation mark at the end: "The

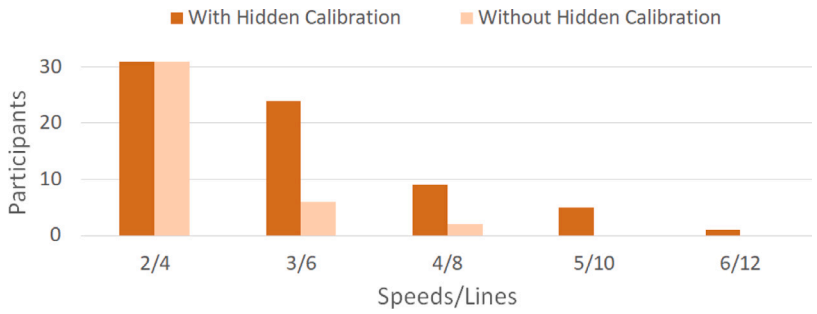


Fig. 17. Test on number of evenly distributed speeds without position reference — Number of participants who completed the test for each number of speeds/lines, with and without hidden calibration.

Table 1
Sentence writing: mean and standard deviation for all the metrics.

	WT [mm:ss]	CPM	WPM	WG	ERR	KSPC	TER
Mean	06:57	6.58	1.32	0.06	0.02	1.16	0.14
STD	01:24	1.29	0.26	0.04	0.02	0.11	0.10

Table 2
Digit entry: mean and standard deviation for WT, CPM, and WPM.

	WT [mm:ss]	CPM	WPM
Mean	00:27	13.77	2.75
STD	00:05	2.09	0.42

Table 3
Digit entry: median and interquartile range for ERR, KSPC, and TER.

	ERR	KSPC	TER
Median	0	1	0
IQR	0.08	0.17	0.15

Table 4
Sentence writing: median and interquartile range for ERR, KSPC, and TER.

	ERR	KSPC	TER
Median	0.02	1.14	0.13
IQR	0.02	0.15	0.13

quick brown fox jumps over the lazy dog!". Then, the participant had to enter two sequences of six digits each, randomly chosen and without digit repetition. In case of errors, the participant had to make corrections.

Indicating with *tc* the total number of characters/digits to write, we assessed the participants' performance through the following metrics:

- *Writing Time* (WT): total time needed to write *tc* characters.
- *Characters Per Minute* (CPM): number of characters written in one minute.
- *Words Per Minute* (WPM): number of words written in one minute, with an average word length of five characters (Arif and Stuerzlinger, 2009).
- *Wrong Group selection* (WG): normalized number of groups of characters wrongly selected (total number divided by *tc*).
- *typing ERRors* (ERR): fraction of wrong characters written, i.e., normalized number of elements wrongly selected (total number divided by *tc*).
- *Key Strokes Per Character* (KSPC) and *Total Error Rate* (TER), as defined by Soukoreff and MacKenzie (2003) (Eqs. (3) and (4)), where C, INF, and IF are, respectively, the number of correct, incorrect not fixed, and incorrect fixed characters, and F is the number of "actions" necessary to fix a wrong character.

$$KSPC = \frac{C + INF + IF + F}{C + INF} \tag{3}$$

$$TER = \frac{INF + IF + F}{C + INF + IF} \tag{4}$$

Table 1 shows mean and standard deviation of all these metrics for the sentence writing test. Regarding the digit entry test, Table 2 shows mean and standard deviation of WT, CPM, and WPM, while Table 3 shows median and interquartile range of ERR, KSPC, and TER — the normality assumption (assessed through the Kolmogorov–Smirnov test) was not satisfied for the distributions of these data.

With the sentence, an average WPM of 1.32 was obtained. With the digit sequence, which only required one selection screen, WPM increased substantially, achieving 2.75. In both cases, we detected very few errors, and KSPC and TER were both close to the ideal values (i.e., 1 for KSPC and 0 for TER).

In statistical terms, paired samples t-tests showed that the difference between the sentence writing and digit entry tests (95% confidence level) is:

- Significant for WT: $t(23) = 22.7$, $p < .001$
- Significant for CPM (and therefore also for WPM): $t(23) = -17.1$, $p < .001$

To compare the performance related to ERR, KSPC, and TER, we also calculated median and interquartile range for the sentence writing test (Table 4). Wilcoxon signed ranks tests showed that the difference between the sentence writing and the digit entry tests is:

- Not significant for ERR: $Z = .92$, $p = .36$
- Significant for KSPC: $Z = 3.39$, $p = .001$
- Significant for TER: $Z = 3.43$, $p = .001$

As might be expected, the use of a single screen for input (digit sequence test) allows for significantly better performance. However, the error rate does not actually change and is extremely low in both cases.

5. Discussion

In Section 2 we have seen that text entry systems based on smooth pursuit are very few. Among these, the first can be considered SMOOVS (Lutz et al., 2015), which was also taken as a model for the solutions by Abdrabou et al. (2019), Zeng et al. (2022), Zeng and Roetting (2018) — however, this last work has been excluded from our analysis since its focus is mainly on the use of a CNN word prediction feature, which is not of interest for our paper. Other recent works exploiting smooth pursuit are *EyeTell* (Bafna et al., 2021), developed for tablets and not using an eye tracker, and *SPEye* (Porta et al., 2022), characterized by a sunburst layout.

The methods proposed by Lutz et al. (2015), Abdrabou et al. (2019), Bafna et al. (2021), and Zeng et al. (2022) implemented a

Table 5
Comparison between Leyenes and other smooth pursuit techniques.

	# Part.[a]	WPM	ERR	MSD	KSPC
SMOOVS (Lutz et al., 2015)	24	3.34	0.18	–	–
(Abdrabou et al., 2019)[c]	26	3.41	–	0.03	1.04
EyeTell (Bafna et al., 2021)	36	1.27	0.14	–	–
SPEye (Porta et al., 2022)	32	0.85	0[b]	–	1[b]
Zeng et al. (2022)	29	4.7	–	0.1	1.67
Leyenes (sentence writing)	31	1.32	0.02	–	1.16
Leyenes (digit entry)	31	2.75	0[b]	–	1[b]

[a] Number of valid participants.
[b] Median instead of mean.
[c] Word prediction enabled.

limited QWERTY keyboard including only the alphabet letters and basic punctuation. Only *SPEye* (Porta et al., 2022) and Leyenes include all the characters of a QWERTY keyboard, with punctuation symbols, accented letters (for Italian and other languages), and numbers, thus allowing a more complete writing process.

Regarding the participants in the experiments, all approaches described in the literature considered a similar number of subjects (between 24 and 36). More precisely, both Lutz et al. (2015) and Abdrabou et al. (2019) involved young participants (25.4 years on average in the first study, 21.6 in the second), with few subjects with corrected-to-normal vision (8 out of 24 in the first study, 8 out of 26 in the second). Zeng et al. (2022) involved 29 participants aged 26.7 years on average, 43% of whom wearing vision aids. Bafna et al. (2021) tested *EyeTell* with 36 participants aged between 18 and 47, but no information was provided about vision correction. Both *SPEye* (Porta et al., 2022) and our current proposal Leyenes were tested with participants with a relatively high average age and with a high percentage of subjects using visual aids: respectively, for *SPEye*, 32 subjects aged between 19 and 71 (30.47 on average), with 19 wearing glasses or contact lenses; for Leyenes, 33 participants (31 valid) aged between 16 and 73 (36 on average), with 19 wearing glasses and 4 contact lenses.

Since each approach adopts different metrics, test conditions, and eye trackers, a perfectly fair comparison between them is not possible. Table 5 reports only the metrics indicated by the authors of the different methods.

The best result obtained with SMOOVS (Lutz et al., 2015) was 3.34 WPM. Normalizing the average number of corrections performed (10.63) according to the number of characters composing the sentence in their experiment (59), we obtain an ERR of 0.18. The alternative configuration with minimum typing errors reached 2.9 WPM and an ERR of 0.14. In comparison, Leyenes is slower, but has also a significantly lower value of ERR.

In the study by Abdrabou et al. (2019), the participants were asked to write five sentences. The average WPM obtained was 3.41, but with word prediction, as no results excluding this feature were provided. The MSD (*Minimum String Distance*) error rate was 3.04%. Given that $MSD = INF/(C + INF)$ (Soukoreff and MacKenzie, 2003), with C and INF as defined in Section 4.4, for Leyenes MSD is 0. Also in this case, Leyenes is slower but is characterized by a lower error rate.

The experiments carried out for *EyeTell* (Bafna et al., 2021) required the participants to write words for a total of 38 characters. The average WPM achieved was 1.27, similar to that of Leyenes. A mean corrected error rate (practically equivalent to ERR) of 0.14 was also reported, significantly higher than ours.

In our comparison we also include the method by Zeng et al. (2022), even if smooth pursuit is a rather limited part of its gaze-based writing process. The user must first select (by looking at it) a cluster of four characters in a set of circularly arranged clusters, and then the four characters pertaining to the observed group start moving in the four directions (horizontally and vertically), riding short paths: it is in this brief phase that smooth pursuit comes into play. Cluster pointing is made possible by an explicit initial one-point calibration procedure. As can be seen from Table 5, the writing speed is very high (4.7 WPM), but the value of MSD is 0.1 (for Leyenes it is 0) and the value of KSPC is 1.67 (1.16 for Leyenes in the sentence writing test).

Finally, *SPEye* (Porta et al., 2022) can be considered the smooth pursuit method with the performance closest to that of Leyenes, although Leyenes is faster (1.32 WPM instead of 0.85 WPM of *SPEye*). The ERR and KSPC values are slightly higher for Leyenes, but the difference is essentially negligible. The speed of Leyenes is also higher when writing digits. Even if digit entry for *SPEye* was evaluated in a test that involved only four participants, the comparison is in favor of Leyenes: the average speed for *SPEye* was 2.1 WPM, as opposed to 2.75 WPM of Leyenes.

Summarizing, Leyenes is slower than the solutions proposed by Lutz et al. (2015), Abdrabou et al. (2019), Bafna et al. (2021), and Zeng et al. (2022), but it is also characterized by less errors, and thus is more robust — the only exception is for the method by Abdrabou et al. (2019), whose results are however provided only with the word prediction functionality active. As stressed in the Introduction, robustness was our main goal, as it had been for *SPEye* (Porta et al., 2022) before. In other words, we wanted to favor accuracy over speed. A low error rate also means that the interaction modality devised provides effective visual feedback, which make errors evident — and, consequently, also facilitate their correction. When dealing, for example, with codes, passwords, or PINs, a low typing error rate is essential. As regards *SPEye*, Leyenes is faster while maintaining comparable values of ERR and KSPC. The preliminary tests (Section 4.1) have shown that, by choosing a speed of 375 px/s, (9.9°/s, neither too high nor too low), Leyenes can be used robustly.

Our study also wanted to investigate the structural parameters of a writing system based on linear smooth pursuit, in which the targets are elements that move horizontally (Section 4.2). These parameters are the number of lines and the number of speeds of the elements moving on the lines. Velocity is a novelty factor that characterizes our study — as far as we know, there are no other works that have considered this parameter in interactions based on smooth pursuit. The absence of an explicit calibration makes it difficult not only to identify the gaze position relative to lines corresponding to different inputs, but also to estimate gaze velocity. This is the reason why the use of more than two speeds is not recommended, and a hidden calibration that allows a rough identification of the vertical position of the gaze is helpful. Section 4.3 showed that both approximate gaze position and speed are important, and their combination resulted in the optimal choice of two speeds and (up to) 12 horizontal lines as input elements. Considering also the two vertical lines in the *Main* screen, Leyenes has a maximum of 14 selectable elements per screen, a number similar to, or higher than, the maximum number of selectable elements used by methods with a circular design — e.g., 6 for SMOOVS (Lutz et al., 2015), 8 for Zeng et al. (2022), and 10–14 (depending on sub-screens) for *SPEye* (Porta et al., 2022).

We believe that, in our reference context (i.e., a gaze-based writing system intended for extemporaneous use in public places), robustness to errors is of paramount importance. Considering long texts, or writing repeated over time, a "learning factor" would come into play which would probably make the difference in robustness between Leyenes and other smooth pursuit systems less evident — but in that case it would presumably be better to use methods other than smooth pursuit, since the initial calibration would not be a significant problem. While word prediction (which is also available in Leyenes) could speed up writing ordinary text, it would not help with codes, PINs, or arbitrary character sequences in general.

6. Conclusion

In this paper we have presented Leyenes, a gaze-based text entry method that exploits smooth pursuit. Our guiding principle was to

design a system that does not require explicit calibration but is nevertheless precise and very little prone to errors. We think that robustness is a crucial feature if we want to employ eye tracking technology in everyday life. As shown in Section 5, compared to the few other writing solutions (totally or partially) based on smooth pursuit, L*eye*nes saves the user from frequent corrections of errors. This feature is achieved at the expense of writing speed, which makes L*eye*nes less suitable for writing long text but helpful in all those cases where the user needs to enter short text, such as codes or numbers. Considering this scenario, we designed a basic interface and a simple interacting modality, so that the application can be quickly understood and used.

The absence of an explicit and precise calibration can also be useful in assistive contexts, whenever a good calibration cannot be achieved, or the user cannot stay sufficiently still in front of the eye tracking device.

Finally, our user study has shown that a linear design, in combination with approximate gaze position and speed estimation, can be a viable alternative to the circular design commonly adopted by smooth pursuit-based writing methods. Our participants were able to correctly select up to 12 horizontally moving elements and 2 vertically moving elements with a very low error rate, whereas methods adopting circular designs generally employ 6 to 10 elements. This is an interesting result that we think can be useful for researchers in the field of gaze-based writing, potentially inspiring the exploration of other configurations.

Future work will further investigate the role that target speed can play in smooth pursuit interaction, maybe introducing the acceleration factor as well. We also plan to explore additional line orientations, besides horizontal and vertical, as well as different layouts.

CRediT authorship contribution statement

Piercarlo Dondi: Methodology, Validation, Data curation, Writing – original draft, Writing – review & editing. **Samuel Sapuppo:** Software, Data curation, Validation, Writing – original draft. **Marco Porta:** Conceptualization, Methodology, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Abdrabou, Y., Mostafa, M., Khamis, M., Elmougy, A., 2019. Calibration-free text entry using smooth pursuit eye movements. In: Proc. ETRA 2019 11th ACM Symposium on Eye Tracking Research & Applications. ACM, New York, NY, USA, pp. 1–5. http://dx.doi.org/10.1145/3314111.3319838.

Arif, A.S., Stuerzlinger, W., 2009. Analysis of text entry performance metrics. In: Proc. 2009 IEEE Toronto International Conference Science and Technology for Humanity. TIC-STH, pp. 100–105. http://dx.doi.org/10.1109/TIC-STH.2009.5444533.

Bafna, T., 2018. Gaze typing using multi-key selection technique. In: Proc. 20th International ACM SIGACCESS Conference on Computers and Accessibility. ACM, New York, NY, USA, pp. 477–479. http://dx.doi.org/10.1145/3234695.3240992.

Bafna, T., Bækgaard, P., Paulin Hansen, J.P., 2021. EyeTell: Tablet-based calibration-free eye-typing using smooth-pursuit movements. In: Proc. ACM Symposium on Eye Tracking Research and Applications. ACM, New York, NY, USA, pp. 1–6. http://dx.doi.org/10.1145/3448018.3458015.

Bee, N., André, E., 2008. Writing with your eye: A dwell time free writing system adapted to the nature of human eye gaze. In: André, E., Dybkjær, L., Minker, W., Neumann, H., Pieraccini, R., Weber, M. (Eds.), Proc. Perception in Multimodal Dialogue Systems. Springer, Berlin, Heidelberg, pp. 111–122. http://dx.doi.org/10.1007/978-3-540-69369-713.

Benligiray, B., Topal, C., Akinlar, C., 2019. SliceType: fast gaze typing with a merging keyboard. J. Multimodal User Interfaces 13 (4), 321–334. http://dx.doi.org/10.1007/s12193-018-0285-z.

Casarini, M., Porta, M., Dondi, P., 2020. A gaze-based web browser with multiple methods for link selection. In: Proc. ETRA 2020 ACM Symposium on Eye Tracking Research and Applications - Adjunt. ACM, New York, NY, USA, pp. 1–8. http://dx.doi.org/10.1145/3379157.3388929.

Cecotti, H., 2016. A multimodal gaze-controlled virtual keyboard. IEEE Trans. Hum.-Mach. Syst. 46 (4), 601–606. http://dx.doi.org/10.1109/THMS.2016.2537749.

Cui, W., Liu, R., Li, Z., Wang, Y., Wang, A., Zhao, X., Rashidian, S., Baig, F., Ramakrishnan, I., Wang, F., Bi, X., 2023. GlanceWriter: Writing text by glancing over letters with gaze. In: Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems. CHI '23, Association for Computing Machinery, New York, NY, USA, http://dx.doi.org/10.1145/3544548.3581269.

Davanzo, N., Dondi, P., Mosconi, M., Porta, M., 2018. Playing music with the eyes through an isomorphic interface. In: Proc. COGAIN '18, the 2018 Symposium on Communication By Gaze Interaction. ACM, New York, NY, USA, pp. 5:1–5:5. http://dx.doi.org/10.1145/3206343.3206350.

Di Maio, M., Dondi, P., Lombardi, L., Porta, M., 2021. Hybrid manual and gaze-based interaction with a robotic arm. In: 2021 26th IEEE International Conference on Emerging Technologies and Factory Automation. ETFA, pp. 1–4. http://dx.doi.org/10.1109/ETFA45728.2021.9613371.

Dondi, P., Porta, M., Donvito, A., Volpe, G., 2022. A gaze-based interactive system to explore artwork imagery. J. Multimodal User Interfaces 16 (1), 55–67. http://dx.doi.org/10.1007/s12193-021-00373-z.

Drewes, H., Khamis, M., Alt, F., 2019. DialPlates: Enabling pursuits-based user interfaces with large target numbers. In: Proceedings of the 18th International Conference on Mobile and Ubiquitous Multimedia. MUM '19, Association for Computing Machinery, New York, NY, USA, http://dx.doi.org/10.1145/3365610.3365626.

Drewes, H., Sakel, S., Hussmann, H., 2022. User perception of smooth pursuit target speed. In: 2022 Symposium on Eye Tracking Research and Applications. ETRA '22, Association for Computing Machinery, New York, NY, USA, http://dx.doi.org/10.1145/3517031.3529234.

Hansen, D.W., Skovsgaard, H.H.T., Hansen, J.P., Møllenbach, E., 2008. Noise tolerant selection by gaze-controlled pan and zoom in 3D. In: Proc. ETRA 2008 Symposium on Eye Tracking Research & Applications. ACM, New York, NY, USA, pp. 205–212. http://dx.doi.org/10.1145/1344471.1344521.

Heikkilä, H., Räihä, K.-J., 2009. Speed and accuracy of gaze gestures. J. Eye Movem. Res. 3 (2), 1:1–14. http://dx.doi.org/10.16910/jemr.3.2.1.

Huang, L., Westin, T., Eladhari, M.P., Magnússon, S., Chen, H., 2023. Eyes can draw: A high-fidelity free-eye drawing method with unimodal gaze control. Int. J. Hum.-Comput. Stud. 170, 102966. http://dx.doi.org/10.1016/j.ijhcs.2022.102966.

Huckauf, A., Urbina, M., 2007. Gazing with pEYE: New concepts in eye typing. In: Proc. 4th Symposium on Applied Perception in Graphics and Visualization. ACM, New York, NY, USA, p. 141. http://dx.doi.org/10.1145/1272582.1272618.

Istance, H., Hyrskykari, A., Immonen, L., Mansikkamaa, S., Vickers, S., 2010. Designing gaze gestures for gaming: An investigation of performance. In: Proc. ETRA 2010, Symposium on Eye-Tracking Research & Applications. ACM, New York, NY, USA, pp. 323–330. http://dx.doi.org/10.1145/1743666.1743740.

Johansen, A.S., Hansen, J.P., Hansen, D.W., Itoh, K., Mashino, S., 2003. Language technology in a predictive, restricted on-screen keyboard with dynamic layout for severely disabled people. In: Proceedings of the 2003 EACL Workshop on Language Modeling for Text Entry Methods. TextEntry '03, Association for Computational Linguistics, USA, pp. 59–66.

Khamis, M., Alt, F., Bulling, A., 2015. A field study on spontaneous gaze-based interaction with a public display using pursuits. In: Adjunct Proceedings of the 2015 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2015 ACM International Symposium on Wearable Computers. In: UbiComp/ISWC'15 Adjunct, Association for Computing Machinery, New York, NY, USA, pp. 863–872. http://dx.doi.org/10.1145/2800835.2804335.

Kumar, C., Menges, R., Müller, D., Staab, S., 2017. Chromium based framework to include gaze interaction in web browser. In: Proc. 26th International Conference on World Wide Web Companion. ACM, International World Wide Web Conferences Steering Committee, pp. 219–223. http://dx.doi.org/10.1145/3041021.3054730.

Kurauchi, A., Feng, W., Joshi, A., Morimoto, C., Betke, M., 2016. EyeSwipe: Dwell-free text entry using gaze paths. In: Proc. CHI 2016 Conference on Human Factors in Computing Systems. ACM, New York, NY, USA, pp. 1952–1956. http://dx.doi.org/10.1145/2858036.2858335.

Li, C., Kim, C., Park, J., 2009. The indirect keyboard control system by using the gaze tracing based on haar classifier in OpenCV. In: Proc. 2009 International Forum on Information Technology and Applications. Vol. 2. pp. 362–366. http://dx.doi.org/10.1109/IFITA.2009.276.

Lutz, O.H.-M., Venjakob, A.C., Ruff, S., 2015. SMOOVS: Towards calibration-free text entry by gaze using smooth pursuit movements. J. Eye Movem. Res. 8 (1), 2: 1–11. http://dx.doi.org/10.16910/jemr.8.1.2.

MacKenzie, I.S., Ashtiani, B., 2011. BlinkWrite: Efficient text entry using eye blinks. Univ. Access Inf. Soc. 10 (1), 69–80. http://dx.doi.org/10.1007/s10209-010-0188-6.

Majaranta, P., Aoki, H., Donegan, M., Witzner Hansen, D., Hansen, J.P., 2011. Gaze Interaction and Applications of Eye Tracking: Advances in Assistive Technologies. IGI Global.

Majaranta, P., Räihä, K.-J., 2007. Text entry by gaze: Utilizing eye-tracking. In: Text Entry Systems: Mobility, Accessibility, Universality. Morgan Kaufmann, San Francisco, pp. 175–187.

Mott, M.E., Williams, S., Wobbrock, J.O., Morris, M.R., 2017. Improving dwell-based gaze typing with dynamic, cascading dwell times. In: Proc. CHI 2017 Conference on Human Factors in Computing Systems. ACM, New York, NY, USA, pp. 2558–2570. http://dx.doi.org/10.1145/3025453.3025517.

Nyström, M., Andersson, R., Holmqvist, K., Van De Weijer, J., 2013. The influence of calibration method and eye physiology on eyetracking data quality. Behav. Res. Methods 45 (1), 272–288. http://dx.doi.org/10.3758/s13428-012-0247-4.

Panwar, P., Sarcar, S., Samanta, D., 2012. EyeBoard: A fast and accurate eye gaze-based text entry system. In: Proc. 4th International Conference on Intelligent Human Computer Interaction. IEEE, pp. 1–8. http://dx.doi.org/10.1109/IHCI.2012.6481793.

Perfect, E., Hoskin, E., Noyek, S., Davies, T.C., 2020. A systematic review investigating outcome measures and uptake barriers when children and youth with complex disabilities use eye gaze assistive technology. Develop. Neurorehabilitation 23 (3), 145–159. http://dx.doi.org/10.1080/17518423.2019.1600066.

Pi, J., Shi, B.E., 2017. Probabilistic adjustment of dwell time for eye typing. In: Proc. 10th International Conference on Human System Interactions. HSI, pp. 251–257. http://dx.doi.org/10.1109/HSI.2017.8005041.

Porta, M., 2015. A study on text entry methods based on eye gestures. J. Assist. Technol. 9 (1), 48–67. http://dx.doi.org/10.1108/JAT-12-2013-0037.

Porta, M., Dondi, P., Pianetta, A., Cantoni, V., 2022. SPEye: A Calibration-free gaze-driven text entry technique based on smooth pursuit. IEEE Trans. Hum.-Mach. Syst. 52 (2), 312–323. http://dx.doi.org/10.1109/THMS.2021.3123202.

Porta, M., Turina, M., 2008. Eye-S: A full-screen input modality for pure eye-based communication. In: Proc. ETRA 2008 Symposium on Eye Tracking Research & Applications. ACM, New York, NY, USA, pp. 27–34. http://dx.doi.org/10.1145/1344471.1344477.

Rajanna, V., 2016. Gaze typing through foot-operated wearable device. In: Proc. 18th International ACM SIGACCESS Conference on Computers and Accessibility. ACM, New York, NY, USA, pp. 345–346. http://dx.doi.org/10.1145/2982142.2982145.

Rajanna, V., Russel, M., Zhao, J., Hammond, T., 2022. PressTapFlick: Exploring a gaze and foot-based multimodal approach to gaze typing. Int. J. Hum.-Comput. Stud. 161, 102787. http://dx.doi.org/10.1016/j.ijhcs.2022.102787.

Shi, L., Copot, C., Vanlanduit, S., 2019. Application of visual servoing and eye tracking glass in human robot interaction: A case study. In: 2019 23rd International Conference on System Theory, Control and Computing. ICSTCC, pp. 515–520. http://dx.doi.org/10.1109/ICSTCC.2019.8886064.

Soukoreff, R.W., MacKenzie, I.S., 2003. Metrics for text entry research: An evaluation of MSD and KSPC, and a new unified error metric. In: Proc. SIGCHI '03 Conference on Human Factors in Computing Systems. ACM, New York, NY, USA, pp. 113–120. http://dx.doi.org/10.1145/642611.642632.

Valencia, S., Lamb, D., Williams, S., Kulkarni, H.S., Paradiso, A., Ringel Morris, M., 2019. Dueto: Accessible, gaze-operated musical expression. In: The 21st International ACM SIGACCESS Conference on Computers and Accessibility. ASSETS '19, Association for Computing Machinery, New York, NY, USA, pp. 513–515. http://dx.doi.org/10.1145/3308561.3354603.

Vidal, M., Bulling, A., Gellersen, H., 2013. Pursuits: Spontaneous interaction with displays based on smooth pursuit eye movement and moving targets. In: Proceedings of the 2013 ACM International Joint Conference on Pervasive and Ubiquitous Computing. UbiComp '13, Association for Computing Machinery, New York, NY, USA, pp. 439–448. http://dx.doi.org/10.1145/2493432.2493477.

Ward, D.J., Blackwell, A.F., MacKay, D.J., 2002. Dasher: A gesture-driven data entry interface for mobile computing. Hum.–Comput. Interact. 17 (2–3), 199–228.

Wobbrock, J.O., Rubinstein, J., Sawyer, M.W., Duchowski, A.T., 2008. Longitudinal evaluation of discrete consecutive gaze gestures for text entry. In: Proc. ETRA 2008 Symposium on Eye Tracking Research & Applications. ACM, New York, NY, USA, pp. 11–18. http://dx.doi.org/10.1145/1344471.1344475.

Zeng, Z., Liu, S., Cheng, H., Liu, H., Li, Y., Feng, Y., Siebert, F., 2023. GaVe: A webcam-based gaze vending interface using one-point calibration. J. Eye Movem. Res. 16 (1), http://dx.doi.org/10.16910/jemr.16.1.2.

Zeng, Z., Neuer, E.S., Roetting, M., Siebert, F.W., 2022. A one-point calibration design for hybrid eye typing interface. Int. J. Hum.–Comput. Interact. 1–14. http://dx.doi.org/10.1080/10447318.2022.2101186.

Zeng, Z., Roetting, M., 2018. A text entry interface using smooth pursuit movements and language model. In: Proc. ETRA 2018 ACM Symposium on Eye Tracking Research & Applications. ACM, New York, NY, USA, pp. 1–2. http://dx.doi.org/10.1145/3204493.3207413.

Zeng, Z., Siebert, F.W., Venjakob, A.C., Roetting, M., 2020. Calibration-free gaze interfaces based on linear smooth pursuit. J. Eye Movem. Res. 13 (1), http://dx.doi.org/10.16910/jemr.13.1.3.